

Pabna University of Science and Technology



Faculty of Engineering and Technology

Department of Information and Communication Engineering

Lab Report

Course Title: System Analysis and Software Testing Sessional

Course Code: ICE-4204

Submitted By:

MD. Mahamudul Hasan

Roll: 200609

Session: 2019-2020

4th Year 2nd semester

Department of Information and
Communication Engineering,

PUST.

Submitted To:

Md. Anwar Hossain

Professor

Department of Information and
Communication Engineering,

PUST.

Date of Submission: 21/05/2025

Signature

INDEX

Pb. No	Problem Name	Page No.
01	Write a program in "JAVA" or "C" to develop a simple calculator that would be able to take a number, an operator (addition/subtraction/multiplication/ division/modulo) and another number consecutively as input and the program will display the output after pressing "=" sign. Sample input: 1+2; 8%4; Sample output: 1+2=3; 8%4=0.	
02	Write a program in "JAVA" or "C" that will take two 'n' integers as input until a particular operator and produce 'n' output. Sample input: 4 5 7 8 20 40 +; Sample output: 9 15 60.	
03	Write a program in "JAVA" or "C" to check weather a number or string is palindrome or not. N.B: your program must not take any test case number such as 1 or 2 for the desired cases from the user. Program user will insert a number or string as input directly and the program will display the exact result in the output console.	
04	Write down the ATM system specifications and report the various bugs.	
05	Write a program in "JAVA" or "C" to find out the factorial of a number using while or for loop. Also verify the results obtained from each case.	
06	Write a program in "JAVA" or "C" that will find sum and average of array using do while loop and 2 user defined function.	
07	Write a simple "JAVA" program to explain classNotFound Exception and endOfFile(Eof) exception.	
08	Write a program in "JAVA" or "C" that will read a input.txt file containing n positive integers and calculate addition, subtraction, multiplication and division in separate output.txt file. Sample input: 5 5 9 8; Sample output: Case-1:10 0 25 1; Case-2: 17 1 72 1.	
09	Explain the role of software engineering in Biomedical Engineering and in the field of Artificial Intelligence and Robotics.	
10	Study the various phases of Water-fall model. Which phase is the most dominated one?	
11	Using COCOMO model estimate effort for specific problem in industrial domain	
12	Identify the reasons behind software crisis and explain the possible solutions for the following scenario: Case 1: "Air ticket reservation software was delivered to the customer and was installed in an airport 12.00 AM (mid night) as per the plan. The system worked quite fine till the next day 12.00 PM (noon). The system crashed at 12.00 PM and the airport authorities could not continue using software for ticket reservation till 5.00 PM. It took 5 hours to fix the defect in the software". Case 2: "Software for financial systems was delivered to the customer. Customer conformed the development team about a mal-function in the system. As the software was huge and complex, the development team could not identify the defect in the software".	

Problem No: 01

Problem Name: Write a program in "JAVA" or "C" to develop a simple calculator that would be able to take a number, an operator (addition/subtraction/multiplication/ division/modulo) and another number consecutively as input and the program will display the output after pressing "=" sign. Sample input: 1+2; 8%4; Sample output: 1+2=3; 8%4=0.

Objective:

To develop a simple calculator program in C or Java that takes two numbers and an operator as input in a single expression and displays the result after evaluation.

Theory: A calculator is a device or program that performs arithmetic operations on numbers. In programming, a simple calculator can be implemented by taking two operands and an operator as input, then performing the corresponding arithmetic operation.

The basic arithmetic operations include:

- **Addition (+):** Adds two numbers.
- **Subtraction (-):** Subtracts the second number from the first.
- **Multiplication (*):** Multiplies two numbers.
- **Division (/):** Divides the first number by the second number (division by zero must be handled).
- **Modulo (%):** Finds the remainder when the first number is divided by the second.

In programming languages like Java and C, these operations are supported by built-in operators. The program should:

1. Take input from the user in the form of an expression (e.g., 1+2).
2. Parse the input to identify the two numbers and the operator.
3. Perform the corresponding arithmetic operation.
4. Display the result in the format 1+2=3.

This experiment helps in understanding:

- How to read and parse user input.
- How to use conditional statements (if-else or switch-case) to decide which operation to perform.
- How to handle exceptions such as division by zero.

Basic programming constructs and arithmetic operations.

Algorithm:

1. **Start**
2. Read the input expression (e.g., 1+2) as a string.
3. Parse the first number, operator, and second number from the input.
4. Use a conditional structure (if or switch) to:
 - Perform addition if operator is +
 - Perform subtraction if operator is -
 - Perform multiplication if operator is *

- Perform division if operator is /
 - Perform modulo if operator is %
5. Calculate the result based on the operator.
 6. Display the full expression with result (e.g., 1+2=3).
 7. **End**

Source Code:

```
#include <stdio.h>
int main() {
    int num1, num2, result;
    char operator;
    printf("Enter expression (e.g., 1+2): ");
    scanf("%d%c%d", &num1, &operator, &num2);

    switch (operator) {
        case '+': result = num1 + num2; break;
        case '-': result = num1 - num2; break;
        case '*': result = num1 * num2; break;
        case '/':
            if (num2 != 0)
                result = num1 / num2;
            else {
                printf("Division by zero error.\n");
                return 1;
            }
            break;
        case '%':
            if (num2 != 0)
                result = num1 % num2;
            else {
                printf("Modulo by zero error.\n");
                return 1;
            }
            break;
        default:
            printf("Invalid operator.\n");
            return 1;
    }
    printf("%d%c%d=%d\n", num1, operator, num2, result);
    return 0;
}
```

Input: Sample input: 1+2; 8%4

Output:

Sample output: 1+2=3; 8%4=0.

Problem No: 02

Problem Name: Write a program in "C" that will take two 'n' integers as input until a particular operator and produce 'n' output. Sample input: 4 5 7 8 20 40 +; Sample output: 9 15 60.

Objective:

To write a C program that takes two sequences of n integers followed by an arithmetic operator, and performs the specified operation pairwise, producing n results as output.

Theory:

This experiment emphasizes **array manipulation** and **element-wise operations**, which are common in scientific and statistical computations.

Key Concepts Involved:

1. Arrays:

Arrays are used to store multiple values of the same type. In this program, two arrays of integers are used to store n inputs each.

2. Arithmetic Operators:

These include:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulo (%)

3. **Looping Constructs:** Loops (like for or while) are essential for traversing arrays and applying operations on each element.

4. **Input Parsing:** The program reads a single input line and splits it into integers and an operator using string manipulation.

5. **Element-Wise Operations:** Each element of the first array is operated with the corresponding element of the second array using the given operator.

Algorithm:

1. Start

2. Initialize two arrays to store the two sequences of n integers.
3. Read input values until an operator (+, -, *, /, %) is encountered.
4. Count the number of integers and divide them into two equal parts.
5. Store the first half in array A and the second half in array B.
6. Identify the operator at the end of input.
7. For each index i from 0 to n-1:
 - Perform the operation $A[i] \text{ op } B[i]$:
 - + → addition
 - - → subtraction

- * → multiplication
- / → division (check for divide-by-zero)
- % → modulo

8. Print the resulting n outputs.

9. **End**

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int numbers[100];
    int count = 0;
    char input[1000], *token;
    char operator;

    printf("Enter integers followed by an operator (e.g., 4 5 7 8 20 40 +):\n");
    fgets(input, sizeof(input), stdin);
    token = strtok(input, " \n");
    while (token != NULL) {
        if (strchr("+-*/%", token[0]) && strlen(token) == 1) {
            operator = token[0];
            break;
        }
        numbers[count++] = atoi(token);
        token = strtok(NULL, " \n");
    }
    if (count % 2 != 0) {
        printf("Invalid input: odd number of integers.\n");
        return 1;
    }

    for (int i = 0; i < count; i += 2) {
        int a = numbers[i];
        int b = numbers[i + 1];
        int result;
        switch (operator) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/': result = (b != 0) ? a / b : 0; break;
            case '%': result = (b != 0) ? a % b : 0; break;
            default:
```

```
        printf("Invalid operator.\n");
        return 1;
    }
    printf("%d ", result);
}
printf("\n");
return 0;
}
```

Input: Enter integers followed by an operator :

4 5 7 8 20 40 +

Output:

9 15 60.

Problem No: 03

Problem Name: Write a program in "C" to check whether a number or string is palindrome or not. **N.B:** your program must not take any test case number such as 1 or 2 for the desired cases from the user. Program user will insert a number or string as input directly and the program will display the exact result in the output console.

Objective:

To write a C program that directly takes a number or string as input and checks whether it is a **palindrome** (i.e., reads the same forwards and backwards), without asking for test case counts.

Theory: A **palindrome** is a number, word, or sequence that reads the same forward and backward. Examples include words like "madam" or numbers like 121. This concept is frequently used in string and number manipulation tasks in programming.

For Strings:

- A string is a palindrome if the characters at the beginning and end match and continue to match as we move inward.
- For example, "level" is a palindrome because the first and last letters are the same, and the second and second-last are the same.
- **For Numbers:**
- A number is treated as a sequence of digits. To check if a number is a palindrome, it can either be converted to a string or reversed mathematically and compared to the original number.

Algorithm:

1. **Start**
2. Take a string input from the user (treat both numbers and strings as character arrays).
3. Determine the length of the input.
4. Initialize two pointers: one at the start, one at the end.
5. While start < end:
 - Compare characters at both pointers.
 - If any pair doesn't match, it's **not a palindrome** → stop.
 - Move the pointers towards the center.
6. If all characters match, it's a **palindrome**.
7. Display the result.
8. **End**

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isPalindrome(char str[]) {
    int left = 0;
    int right = strlen(str) - 1;

    while (left < right) {
        // Convert both characters to lowercase to make it case-insensitive
        char l = tolower(str[left]);
        char r = tolower(str[right]);

        if (l != r) {
            return 0; // Not a palindrome
        }
        left++;
        right--;
    }
    return 1; // Is a palindrome
}

int main() {
    char input[1000];

    printf("Enter a number or string: ");
    scanf("%s", input);

    if (isPalindrome(input)) {
        printf("%s is a palindrome.\n", input);
    } else {
        printf("%s is not a palindrome.\n", input);
    }

    return 0;
}
```

Input: Enter a number or string: I want to be a good person

Enter a number or string: 4004

Output: I want to be a good person is not a palindrome.

4004 is not a palindrome.

Problem No: 04

Problem Name: Write down the ATM system specifications and report the various bugs.

Objective:

To specify the functional and non-functional requirements of an ATM (Automated Teller Machine) system and identify potential bugs or issues in its operations to improve reliability, security, and user experience.

Theory:

1. System Overview

An Automated Teller Machine (ATM) provides customers with banking services such as cash withdrawals, deposits, balance inquiries, and fund transfers without the need for human interaction.

2. Core Functional Requirements

- **Authentication**
 - Card insertion
 - PIN verification (usually 4 or 6 digits)
- **Transaction Services**
 - Cash Withdrawal
 - Balance Inquiry
 - Mini Statement
 - Fund Transfer (within bank or inter-bank)
 - Deposit (cash or check)
- **User Interface**
 - Multilingual support
 - Touchscreen or button-based navigation
 - Receipt printing option
- **Security**
 - Encrypted PIN and transaction processing
 - Session timeout
 - Camera surveillance (hardware)
- **Admin Services**
 - Cash loading and denomination management
 - System diagnostics
 - Logs of usage and errors

3. Non-Functional Requirements

- **Availability:** 24/7 operation
- **Performance:** Transactions processed within a few seconds
- **Scalability:** Support for integration with multiple banks and accounts

- **Reliability:** Must handle power failures gracefully (e.g., UPS backup)
- **Maintainability:** Logs, alerts, and remote diagnostics

4. Hardware Interfaces

- Card Reader (EMV chip/magnetic strip)
- PIN Pad / Touchscreen
- Cash Dispenser
- Receipt Printer
- Deposit Module
- Network Modem or Router

5. Software Interfaces

- Bank Core System API
- Encryption Libraries
- Operating System (typically Linux or a secure embedded OS)
- Database for local logging

Common Bugs in ATM Systems

1. Transaction Bugs

- **Double Deduction:** Amount is debited twice due to a timeout or retry logic bug.
- **Amount Not Dispensed:** Transaction success is shown but cash isn't dispensed.
- **Wrong Balance Display:** After a transaction, the balance does not reflect accurately.
- **Deposit Not Updated:** Deposited amount doesn't reflect in the account.
- **Mini Statement Inaccuracy:** Transactions are missing or incorrectly ordered.

2. UI/UX Bugs

- **Incorrect Language Switching:** Selecting a language doesn't change the UI language.
- **Unresponsive Buttons:** Hardware or software glitches make screen/buttons unusable.
- **Wrong Instructions:** Prompts don't match the transaction context (e.g., shows "Insert Card" after already inserted).

3. Security Bugs

- **Session Doesn't Expire:** No timeout when the user walks away.
- **PIN Not Masked:** PIN entry visible on the screen.
- **No Encryption in Transmission:** PIN or transaction data sent as plain text.

4. Concurrency Issues

- **Race Condition:** Two withdrawals at the same time from different ATMs deduct more than the available balance.
- **Out-of-Sync Logs:** Transaction log not synchronized with the bank core system.

Problem No: 05

Problem Name: Write a program in "C" to find out the factorial of a number using while or for loop. Also verify the results obtained from each case.

Objective:

To write a C program that calculates the factorial of a given number using both for and while loops, and verifies that both methods produce the same result.

Theory:

Loops (for/while):

- Used to iterate from 1 to n and multiply each value to get the factorial.
- **Integer Variables:**
 - One variable to hold the input number (n).
 - One variable to store the result (fact), initialized to 1.
- **Verification:**
 - Running both for and while loop versions and comparing the outputs confirms correctness.
- **Input Validation:**
 - Factorial is only defined for non-negative integers.

Algorithm (Using For Loop):

1. Start
2. Initialize a variable fact = 1
3. Read input number n
4. If $n < 0$, display "Invalid input"
5. Otherwise,
 - Loop from $i = 1$ to n
 - Multiply $fact = fact * i$
6. Display fact
7. End

Source Code:

```
#include <stdio.h>

int factorialForLoop(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
```

```

        result *= i;
    }
    return result;
}

int factorialWhileLoop(int n) {
    int result = 1;
    int i = 1;
    while (i <= n) {
        result *= i;
        i++;
    }
    return result;
}

int main() {
    int number;
    printf("Enter a positive integer: ");
    scanf("%d", &number);

    if (number < 0) {
        printf("Factorial is not defined for negative numbers.\n");
        return 1;
    }

    int factFor = factorialForLoop(number);
    int factWhile = factorialWhileLoop(number);

    printf("Factorial using for loop: %d\n", factFor);
    printf("Factorial using while loop: %d\n", factWhile);

    if (factFor == factWhile) {
        printf("✅ Both results match. Factorial is %d.\n", factFor);
    } else {
        printf("❌ Mismatch in results! Check the logic.\n");
    }

    return 0;
}

```

Input:

Enter a positive integer: 5

Output:

Factorial using for loop: 120
 Factorial using while loop: 120
 Both results match. Factorial is 120.

Problem No: 06

Problem Name: Write a program in or "C" that will find sum and average of array using do while loop and 2 user defined function.

Objective:

To write a C program that calculates the **sum** and **average** of elements in an array using a do-while loop and two **user-defined functions**—one for computing the sum and another for calculating the average.

Theory:

To find the **sum** and **average** of elements in an array in C:

- An array is a collection of similar data types stored in contiguous memory locations.
- The **sum** is the total of all elements.
- The **average** is the sum divided by the total number of elements.
- The task will be divided into **two functions**:
 1. calculateSum() – computes the sum using a do...while loop.
 2. calculateAverage() – calculates the average using the sum and number of elements.

Algorithm:

Main Function:

1. Declare an array and variables for sum and average.
2. Take input for array size n.
3. Read n array elements from the user.
4. Call findSum() function to get the sum of array elements.
5. Call findAverage() function to compute the average using the sum and n.
6. Display the sum and average.

Function 1: findSum(int arr[], int n)

1. Initialize sum = 0, i = 0
2. Use do-while loop:
 - Add arr[i] to sum
 - Increment i
 - Continue while i < n
3. Return sum

Function 2: findAverage(int sum, int n)

1. Calculate average = (float)sum / n
2. Return average

Source Code:

```
#include <stdio.h>

// Function to calculate sum of array
int calculateSum(int arr[], int n) {
```

```

int sum = 0, i = 0;
do {
    sum += arr[i];
    i++;
} while (i < n);
return sum;
}
// Function to calculate average of array
float calculateAverage(int sum, int n) {
    return (float)sum / n;
}
int main() {
    int arr[100], n, i;

    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid number of elements.\n");
        return 1;
    }
    printf("Enter %d integers:\n", n);
    i = 0;
    do {
        printf("Element %d: ", i + 1);
        scanf("%d", &arr[i]);
        i++;
    } while (i < n);
    int sum = calculateSum(arr, n);
    float average = calculateAverage(sum, n);
    printf("\nSum of array elements = %d\n", sum);
    printf("Average of array elements = %.2f\n", average);
    return 0;
}

```

Input:

Enter the number of elements in the array: 5
 Enter 5 integers:
 Element 1: 5
 Element 2: 8
 Element 3: 9
 Element 4: 8
 Element 5: 2

Output:

Sum of array elements = 32
 Average of array elements = 6.40

Problem No: 07

Problem Name: Write a simple "JAVA" program to explain classNotFound Exception and endOfFile(EOF) exception.

Objective:

To write a simple Java program that demonstrates the use of exception handling for ClassNotFoundException and EOFException, showing how these exceptions occur during dynamic class loading and file input operations, and how to handle them gracefully using try-catch blocks.

Theory:

ClassNotFoundException

- **Definition:**

ClassNotFoundException is a checked exception in Java that occurs when an application tries to load a class dynamically at runtime using methods like Class.forName() or ClassLoader.loadClass(), but the class with the specified name cannot be found in the classpath.

- **When does it occur?**

This exception typically happens when the JVM cannot locate the .class file for the requested class because it is either missing, incorrectly named, or not included in the classpath.

- **Importance:**

Handling this exception is essential in programs that rely on dynamic class loading, such as plugin systems or applications using reflection.

2. EOFException (End of File Exception)

- **Definition:**

EOFException is a subclass of IOException in Java. It signals that an end of file or stream has been reached unexpectedly during input operations, especially when using data streams like DataInputStream or ObjectInputStream.

- **When does it occur?**

This exception occurs when a program tries to read more data from a file or input stream than is available, i.e., it encounters the end of the file or stream prematurely.

- **Importance:**

Catching this exception allows programs to detect when no more data can be read from a stream, enabling graceful termination of reading loops and avoiding infinite waiting or errors.

Source code:

```
import java.io.*;

public class ExceptionDemo {

    public static void demoClassNotFoundException() {
        try {
            // Try to load a class that doesn't exist
            Class.forName("com.example.NonExistentClass");
        } catch (ClassNotFoundException e) {
            System.out.println("Caught ClassNotFoundException: " + e.getMessage());
        }
    }

    public static void demoEOFException() {
        try {
            FileInputStream fis = new FileInputStream("testdata.dat");
            DataInputStream dis = new DataInputStream(fis);

            while (true) {
                // Intentionally read more data than file contains to cause EOFException
                int number = dis.readInt();
                System.out.println("Read number: " + number);
            }
        } catch (EOFException e) {
            System.out.println("Caught EOFException: Reached end of file.");
        } catch (IOException e) {
            System.out.println("IOException: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("Demo: ClassNotFoundException");
        demoClassNotFoundException();
        System.out.println("\nDemo: EOFException");
        demoEOFException();
    }
}
```

Problem No: 08

Problem Name: Write a program in "C" that will read a input.txt file containing n positive integers and calculate addition, subtraction, multiplication and division in separate output.txt file. Sample input: 5 5 9 8; Sample output: Case-1:10 0 25 1; Case-2: 17 1 72 1.

Objective:

To develop a program in **Java** or **C** that reads n positive integers from an input.txt file, performs pairwise **addition**, **subtraction**, **multiplication**, and **division**, and writes the results for each pair to an output.txt file in a structured format.

Theory:

File handling in C allows programs to read data from files and write data to files, enabling persistent data storage. This program reads n positive integers from an input file input.txt and performs arithmetic operations (addition, subtraction, multiplication, and division) on pairs of numbers.

- The program assumes the input contains pairs of numbers. For each pair:
 - Addition: sum of the two numbers.
 - Subtraction: difference (first number minus second).
 - Multiplication: product of the two numbers.
 - Division: integer division (first number divided by second).
- The results for each pair are written to an output file output.txt with appropriate labeling.
- Basic file operations in C involve using fopen, fprintf, fscanf, and fclose.

Algorithm:

1. **Start**
2. Open the input file input.txt in read mode.
 - If file opening fails, display error and terminate.
3. Open the output file output.txt in write mode.
 - If file opening fails, display error and terminate.
4. Read integers from input.txt.
 - Assume the number of integers n is even (pairs of numbers).
 - Alternatively, read until end of file.
5. For every pair of numbers (a, b) read:
 - Calculate:
 - Addition = $a + b$
 - Subtraction = $a - b$
 - Multiplication = $a * b$
 - Division = a / b (integer division; check for division by zero)
6. Write the results to output.txt in the format:

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fin = fopen("input.txt", "r");
    FILE *fout = fopen("output.txt", "w");

    if (fin == NULL) {
        printf("Error opening input.txt\n");
        return 1;
    }
    if (fout == NULL) {
        printf("Error opening output.txt\n");
        fclose(fin);
        return 1;
    }
    int numbers[1000];
    int count = 0;
    while (fscanf(fin, "%d", &numbers[count]) == 1) {
        count++;
    }
    if (count % 2 != 0) {
        fprintf(fout, "Error: Odd number of integers in input file.\n");
        fclose(fin);
        fclose(fout);
        return 1;
    }
    for (int i = 0, case_num = 1; i < count; i += 2, case_num++) {
        int a = numbers[i];
        int b = numbers[i + 1];
        int add = a + b;
        int sub = a - b;
        int mul = a * b;
        int div = (b != 0) ? (a / b) : 0; // handle division by zero gracefully
        fprintf(fout, "Case-%d: %d %d %d %d\n", case_num, add, sub, mul, div);
    }
    fclose(fin);
    fclose(fout);

    printf("Processing complete. Check output.txt\n");

    return 0;
}
```

Input:

Enter the number: 5 5 9 8

Output:

Case-1: 10 0 25 1

Case-2: 17 1 72 1

Problem No: 09

Problem Name: Explain the role of software engineering in Biomedical Engineering and in the field of Artificial Intelligence and Robotics.

Objective:

To understand and explain the crucial role of software engineering in advancing **Biomedical Engineering** and its applications, as well as its impact on the development of **Artificial Intelligence (AI)** and **Robotics**, highlighting how software solutions improve healthcare technologies, intelligent systems, and automation.

Theory:

1. Role of Software Engineering in Biomedical Engineering

A. Medical Software Development

Software engineers develop applications and systems used in hospitals, clinics, and diagnostic labs. Examples include:

- **Electronic Health Records (EHR) systems**
- **Medical imaging software** (e.g., MRI, CT scan analysis)
- **Patient monitoring systems**
- **Decision support systems** for doctors

B. Biomedical Device Control

- Many biomedical devices (like pacemakers, infusion pumps, or prosthetics) are software-driven.
- Software engineers write the **embedded software** that controls these devices.

C. Data Analysis and Signal Processing

- Biomedical signals (like ECG, EEG, EMG) require specialized software to:
 - Filter noise
 - Detect patterns or anomalies
 - Aid in diagnosis (e.g., detecting arrhythmia from ECG)
- Software engineering ensures this is done **efficiently, accurately, and safely**.

D. Regulatory and Safety Compliance

- Software in biomedical systems must meet **strict regulatory standards** (e.g., FDA, ISO 13485).

- Software engineers must document development and perform **extensive testing and validation**.

E. Modeling and Simulation

- Tools like MATLAB or Simulink are used for:
 - Simulating biological systems
 - Testing control algorithms before implementation in physical devices

2. Role of Software Engineering in Artificial Intelligence and Robotics

A. Algorithm Development

- AI systems rely on **complex algorithms** for:
 - Machine learning
 - Natural language processing
 - Computer vision
- Software engineers implement, optimize, and maintain these algorithms in real-world applications.

B. Robotics Control Systems

- Robots need **real-time control systems** to move, sense, and respond.
- Software engineers write:
 - **Firmware** for motor and sensor control
 - **Middleware** like ROS (Robot Operating System)
 - **High-level logic** for navigation, object recognition, etc.

C. Simulation and Testing

- Before deploying AI models or robots in the real world, software engineers:
 - Use **simulators** (e.g., Gazebo, Unity, Webots)
 - Perform **unit testing, integration testing, and stress testing**

D. Ethics and Safety

- Engineers must design systems that are:
 - Safe to operate
 - Explainable and unbiased (especially in AI)
 - Fail-safe in case of hardware or software malfunction

E. Human-Robot Interaction (HRI)

- Software engineers build interfaces that allow humans to interact naturally with robots using:
 - Voice commands

Problem No: 10

Problem Name: Study the various phases of Water-fall model. Which phase is the most dominated one?

Objective:

To study and understand the different phases of the **Waterfall Software Development Model** and identify which phase typically dominates or requires the most effort in the overall software development process.

Theory:

The **Waterfall Model** is one of the earliest and most straightforward software development life cycle (SDLC) models. It is a **linear and sequential approach**, where each phase must be completed before the next begins. Here's an overview of its **phases** and an analysis of the **most dominant phase**.

Phases of the Waterfall Model

1. Requirement Analysis

- Involves gathering and documenting all user and system requirements.
- Outputs a **Software Requirements Specification (SRS)** document.
- No coding or designing happens here—focus is on what the system should do.

2. System Design

- Based on the SRS, system architecture and design are created.
- Includes:
 - High-level design (HLD) – overall system architecture
 - Low-level design (LLD) – module-level details
- Determines programming language, data structures, UI design, etc.

3. Implementation (Coding)

- Developers write the actual source code according to the design documents.
- This phase turns design into a working product (software application).

4. Integration and Testing

- Individual modules are integrated and tested as a complete system.
- Focus is on:
 - Unit testing
 - System testing
 - Acceptance testing
- Objective: detect and fix bugs, verify functionality.

5. Deployment

- The fully tested product is delivered to the customer or deployed in a live environment.

6. Maintenance

- Ongoing support:
 - Bug fixes
 - Updates or enhancements
 - Performance improvements
- Longest-lasting phase in many real-world projects.

Most Dominant Phase: Requirement Analysis

- **Foundation of the entire project:** If requirements are wrong, everything built after will be flawed.
- It **influences all other phases:** design, implementation, testing, etc.
- Waterfall assumes that requirements are fixed early; so **thorough, accurate analysis** is critical.
- Errors in this phase are **costlier to fix later** than those made in coding or testing.

"Poor requirements gathering is the leading cause of software project failure."

Alternative Perspective:

Some may argue that **Maintenance** dominates in terms of time and effort in the long run, especially for large systems. However, from a **process-centric and planning viewpoint**, **Requirement Analysis** holds the most **influence** and is considered the **most critical (and dominant)** in the Waterfall model.

Problem No: 11

Problem Name: Using COCOMO model estimate effort for specific problem in industrial domain.

Objective:

COCOMO is a software estimation model developed by **Barry Boehm** to predict the cost, effort, and schedule of a software project. It uses mathematical formulas based on historical project data.

There are three types of COCOMO models:

1. **Basic COCOMO** – Uses only size (KLOC) to estimate effort.
2. **Intermediate COCOMO** – Adds cost drivers (factors like reliability, complexity, experience).
3. **Detailed COCOMO** – Includes all features of intermediate and more breakdown by phase.

For an **industrial domain**, we commonly use the **Intermediate COCOMO** model to account for complexity and real-world factors.

Theory:

The COCOMO (Constructive Cost Model) model is used to estimate the effort, cost, and schedule for software projects based on the size of the software and various project characteristics. To estimate

The **Basic COCOMO Model** estimates:

- **Effort (in person-months)** using the formula:

$$(PM)Effort (PM)=a \times (KLOC)^b$$

Where:

- KLOC = Thousands of Lines of Code
- a, b = constants based on the project type:
 - **Organic** (simple, small teams, familiar domain)
 - **Semi-detached** (medium-sized, mixed experience)
 - **Embedded** (complex, tightly coupled with hardware or regulations)

Problem Scenario: Industrial Process Monitoring System

Let's assume you're developing a **real-time industrial monitoring system** (e.g., to monitor pressure, temperature, etc., in a chemical plant).

Assumptions:

- Estimated size: **50 KLOC**
- Project Type: **Embedded** (since it's safety-critical and hardware-interfacing)
- Basic COCOMO constants for Embedded:
 - $a = 3.6$, $b = 1.20$

COCOMO Effort Estimation

$$\text{Effort} = 3.6 \times (50)^{1.20}$$

Let's calculate:

$$(50)^{1.20} \approx 109.4 \text{ person-months} \quad \text{Effort} = 3.6 \times 109.4 \approx 393.8 \text{ person-months}$$

Development Time (Optional)

You can also estimate development time:

$$(\text{months}) \text{Time (months)} = c \times (\text{Effort})^d$$

For Embedded: $c = 2.5$, $d = 0.32$

$$\text{monthsTime} = 2.5 \times (393.8)^{0.32} \approx 2.5 \times 7.2 \approx 18 \text{ months}$$

Using the COCOMO model, the estimated effort for a 25 KLOC industrial (embedded) project is approximately 195.5 person-months, and it would take about 14.7 months to complete with an average-size team. These estimates help in planning resources, budgeting, and scheduling in industrial software projects.

Problem No: 12

Problem Name: Identify the reasons behind software crisis and explain the possible solutions for the following scenario:

Case 1: "Air ticket reservation software was delivered to the customer and was installed in an airport 12.00 AM (mid night) as per the plan. The system worked quite fine till the next day 12.00 PM (noon). The system crashed at 12.00 PM and the airport authorities could not continue using software for ticket reservation till 5.00 PM. It took 5 hours to fix the defect in the software".

Case 2: "Software for financial systems was delivered to the customer. Customer conformed the development team about a mal-function in the system. As the software was huge and complex, the development team could not identify the defect in the software".

Objective:

The term "**Software Crisis**" refers to the set of problems faced during the early days of software engineering and still relevant today in large or critical systems. These include difficulties in software development such as project overruns, low quality, poor maintainability, and system failures.

Theory:

The term "Software Crisis" refers to the set of problems faced during the early days of software engineering and still relevant today in large or critical systems.

Reasons Behind the Software Crisis

1. Poor Requirements Analysis

Inadequate understanding of what the software should do leads to incorrect or incomplete functionality.

2. Lack of Design Standardization

Ad hoc designs result in unscalable and hard-to-maintain systems.

3. Inadequate Testing

Systems are released without rigorous testing, especially in edge cases or under real-time conditions.

4. Complexity of Software

As systems become larger, managing interdependencies becomes very difficult.

5. Lack of Proper Documentation

When code is not well-documented, debugging and maintenance become harder.

6. Underestimation of Project Time and Resources

Unrealistic timelines and insufficient staffing lead to rushed, buggy code.

7. Poor Maintenance Planning

Post-delivery issues are not addressed efficiently due to the lack of ongoing support structures.

Case Analysis and Solutions

Case 1: Airport Ticket Reservation System Crash

Issue Summary:

- The system worked fine for 12 hours (from midnight to noon) and then crashed.
- System was unusable for 5 hours.
- This implies a **runtime defect** related to time handling or load handling.

Possible Root Causes:

- **Time Format Bug:** A 12-hour vs 24-hour time format issue (e.g., confusion between AM/PM or time rollover).
- **Memory Leak:** A resource leak that accumulated until system resources were exhausted.
- **Load Threshold Exceeded:** System not tested under real-world traffic, leading to overload.
- **Date-Time Overflow Bug:** A bug in a counter or timestamp function.
- **Insufficient Logging and Monitoring:** Delayed response time in fixing the bug.

Solutions:

1. Rigorous Boundary Testing

- Test edge cases like 11:59 AM to 12:01 PM transitions.
- Validate AM/PM vs 24-hour handling.

2. Load and Stress Testing Before Deployment

- Simulate real-world ticketing loads for extended periods.

3. Automated Monitoring and Logging

- Deploy monitoring tools that alert immediately on abnormal behavior.

4. Fallback and Redundancy Planning

- Deploy redundant systems or failover options to avoid total outages.

5. Hotfix Framework in Place

- Ensure modular patching is possible without system downtime.

Case 2: Financial System Malfunction – Hard to Identify Defect

Issue Summary:

- The system is **large and complex**.
- A defect was reported, but developers were **unable to locate it** easily.

Possible Root Causes:

- **Poor Modularity:** Large monolithic codebase makes defect tracing difficult.
- **Lack of Logging/Tracing:** Inability to reproduce the issue or track its source.
- **Inadequate Documentation:** Developers can't understand how different components interact.
- **Code Complexity:** High cyclomatic complexity or tangled logic.
- **Insufficient Unit/Integration Tests:** Hard to pinpoint which module fails.

Solutions:

1. Modular Design and Architecture

- Follow principles like **Separation of Concerns** and **Single Responsibility**.
- Use microservices where appropriate.

2. Advanced Debugging Tools

- Use profilers, debuggers, and log analyzers to trace defects.

3. Detailed Logging and Auditing

- Implement application-wide logging at various levels (info, warn, error).

4. Test-Driven Development (TDD)

- Writing tests before code improves defect identification and localization.

5. Code Refactoring and Documentation

- Periodically refactor the code to reduce complexity.
- Maintain updated architecture diagrams and module interaction maps.

6. Team Knowledge Sharing

- Conduct walkthroughs and knowledge transfer sessions regularly.

Conclusion

The software crisis emerges mainly from **lack of proper planning, testing, and design practices**. In both cases, adopting **engineering best practices, rigorous testing, modular architecture, and active monitoring** can prevent or mitigate such failures.